

# EEE373L

Embedded System Design Laboratory

## Lab Report



Inspiring Excellence

Section:02

Group No:02

Experiment no:04

**Name of the experiment:** Single and Multiple External Interrupts

*Prepared by:*

**Name:** Md. Hamin-Al-Araf

**ID:** 21221019

**Signature**

*All Group members:*

| Sl. | Name                | ID       | Signature |
|-----|---------------------|----------|-----------|
| 1.  | Yasir Anaf Siddique | 21221073 |           |
| 2.  | Md Talha Bin Imran  | 21221018 |           |
| 3.  | Nafim Karim Khan    | 21221039 |           |
| 4.  | Afrida Islam        | 21221030 |           |

*Date of Submission:* 22/06/2024

**Objective:** The objective of this experiment is to understand the basics of external single interrupts and multiple interrupts.

***Theoretical Background:***

An interrupt is a mechanism that allows a microcontroller to temporarily pause its normal, sequential task execution to address a different, more urgent task. In typical operation, a microcontroller executes a program line by line. However, when a programmer needs to divert the microcontroller's attention to an immediate task, an interrupt is used. During such events, the microcontroller raises a specific flag related to the task. The microcontroller is configured to detect this flag and, using an Interrupt Service Routine (ISR), automatically handles the task whenever the flag is activated, resuming normal operation afterward.

In AVR 8-bit microcontrollers, there are both internal and external sources of interrupts. Internal interrupts are tied to the microcontroller on board peripherals, such as the Timer/Counter or the Analog Comparator. External interrupts, on the other hand, are triggered via dedicated external pins. For example, the AVR ATmega32 microcontroller supports four external interrupts:

- RESET Interrupt: Triggered by pin 9.
- External Interrupt 0 (INT0): Triggered by pin 16.
- External Interrupt 1 (INT1): Triggered by pin 17.
- External Interrupt 2 (INT2): Triggered by pin 3.

This experiment will mainly focus on external interrupts. The first part, PART A will demonstrate the use of a single interrupt and the second part, PART B, will demonstrate the use of multiple interrupts.

## PART A (Single External Interrupt)

### Circuit Diagram

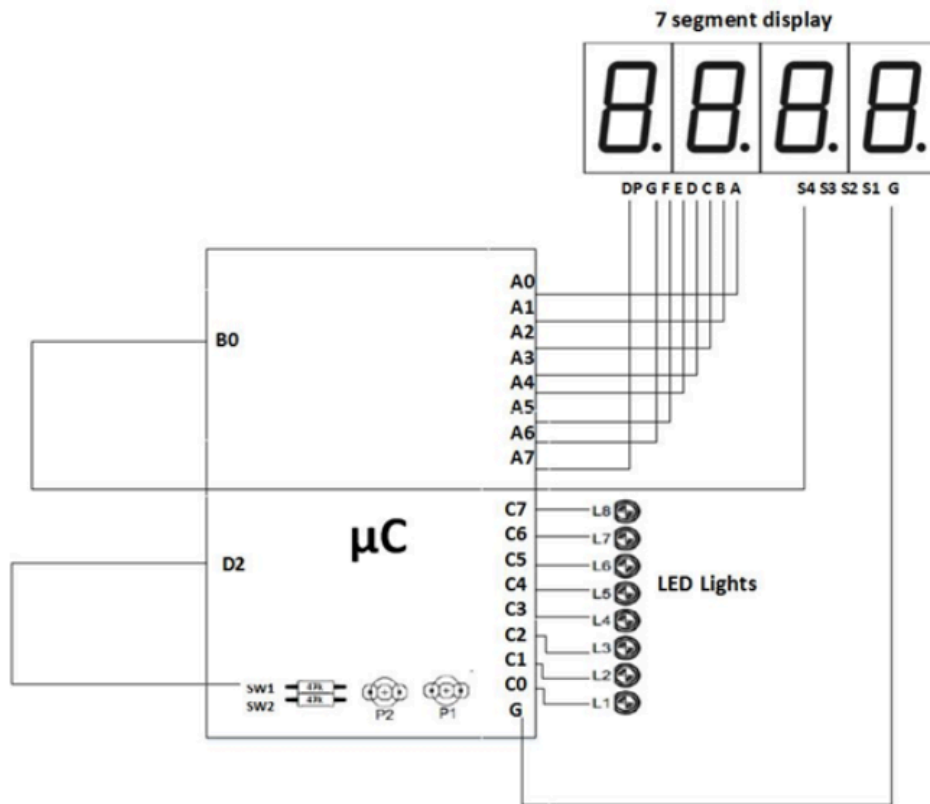


Figure 01: Circuit diagram of Single External Interrupt task

### Code

```
#include <mega32.h>
#include <delay.h>

interrupt [EXT_INT0] void ext_int0_isr(void)
{
    unsigned int cathode[10]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
    unsigned int i=0,k=0;
    PORTB=0x01;
    for (i=0;i<10;i++)
    {
        PORTA= cathode[i];
        delay_ms(1000);
    }
}
```

```

    PORTA=0;
    PORTB=0;
}

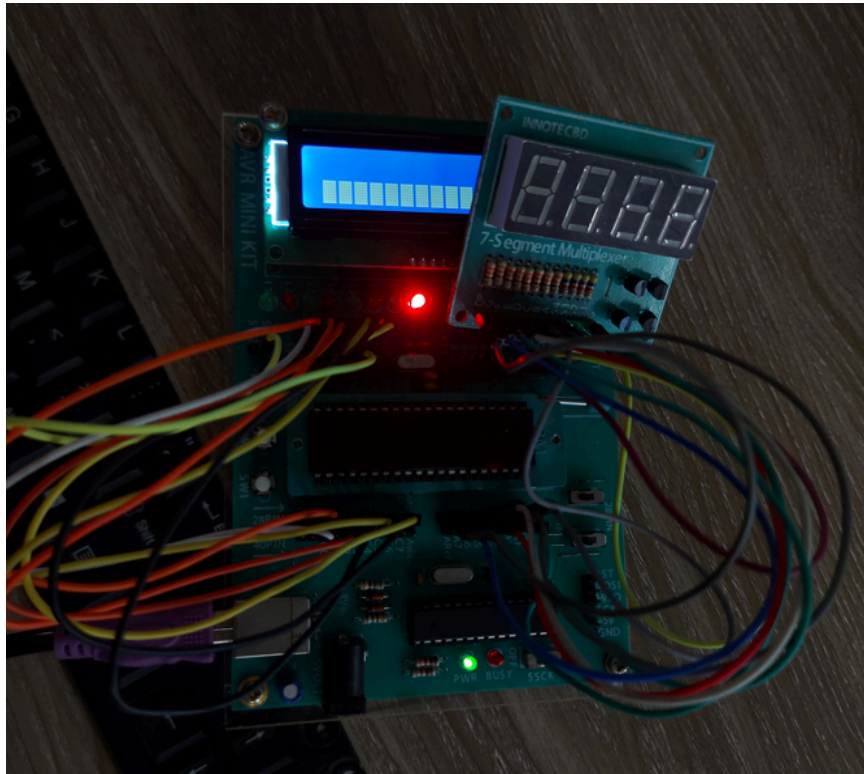
void main(void)
{
    char i;
    DDRA=0xFF;
    DDRC=0xFF;
    DDRB=0x01;
    PORTA=0;
    PORTC=0;
    PORTB=0;

    GICR|=(0<<INT1) | (1<<INT0) | (0<<INT2);
    MCUCR=(0<<ISC11) | (0<<ISC10) | (1<<ISC01) | (0<<ISC00);
    MCUCSR=(0<<ISC2);
    GIFR=(0<<INTF1) | (1<<INTF0) | (0<<INTF2);

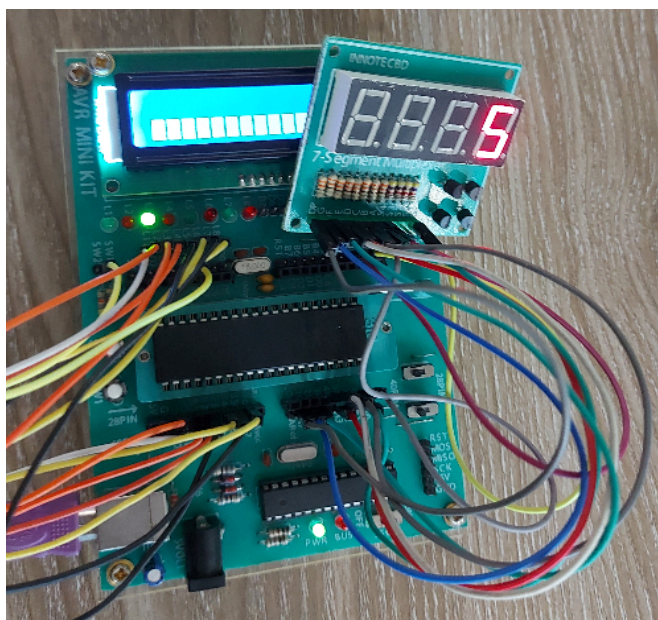
    #asm("sei")
    while (1)
    {
        PORTC=0x80;
        for (i=0;i<8;i++)
        {
            delay_ms(1000);
            PORTC>>=1;
        }
    }
}

```

## *Output*



**Figure 02: LED keeps shifting (Infinite While Loop task)**



**Figure 03: Interrupted Task. 7 Segment display starts showing countdown**

**Discussion:** This task focuses on a single external interrupt. For the interrupt, PORTD.2 (INT0) is used. When the code is run, it executes the task defined in the infinite loop which is the LEDs shifting towards right. When the interrupt button is pressed, the LEDs stop shifting and the 7-segment display starts showing a countdown from 0 to 9 with a 1 second interval. This is how an infinite while loop task is interrupted to do another task using a single interrupt.

## PART B (Multiple external Interrupts)

*Circuit Diagram:*

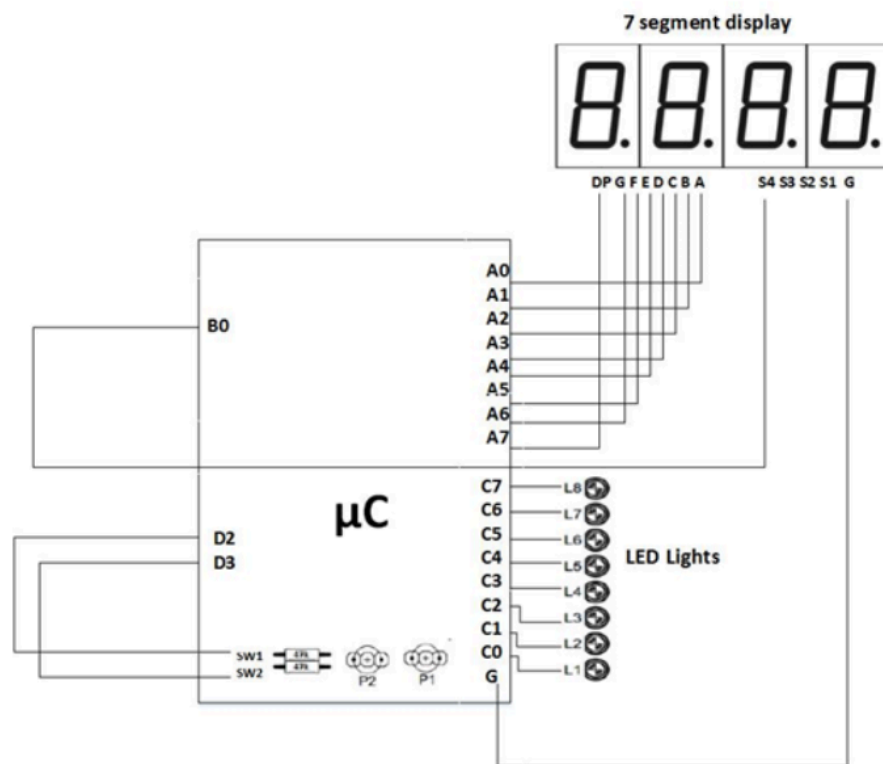


Figure 04: Circuit Diagram of Multiple External Interrupts

*Code*

```
#include <mega32.h>
#include <delay.h>
```

```

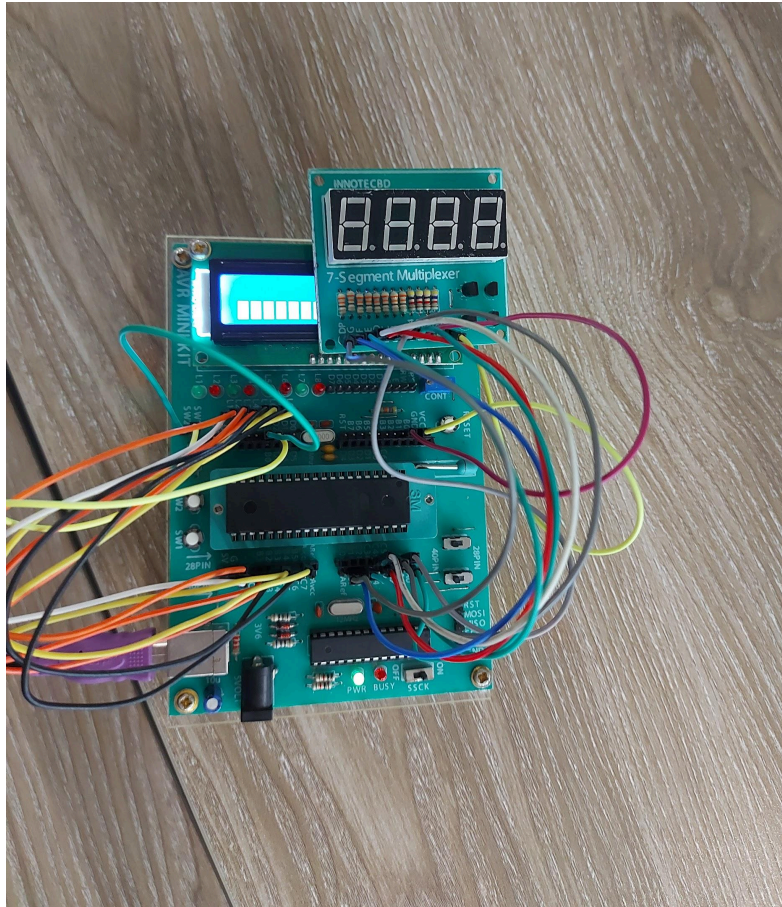
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    unsigned int cathode[10]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
    unsigned int i=0,k=0;
    PORTB=0x01;
    for (i=0;i<10;i++)
    {
        PORTA= cathode[i];
        delay_ms(1000);
    }
    PORTA=0;
    PORTB=0;
}
interrupt [EXT_INT1] void ext_int1_isr(void)
{
    char i;
    #asm("sei")
    PORTC=0x80;
    for (i=0;i<8;i++)
    {
        delay_ms(1000);
        PORTC>>=1;
    }
}
void main(void)
{

    DDRA=0xFF;
    DDRC=0xFF;
    DDRB=0x01;
    PORTA=0;
    PORTC=0;
    PORTB=0;
    GICR|=(1<<INT1) | (1<<INT0) | (0<<INT2);
    MCUCR=(1<<ISC11) | (0<<ISC10) | (1<<ISC01) | (0<<ISC00);
    MCUCSR=(0<<ISC2);
    GIFR=(1<<INTF1) | (1<<INTF0) | (0<<INTF2);
    #asm("sei")
    while (1)
    {
        // Place your code here

    }
}

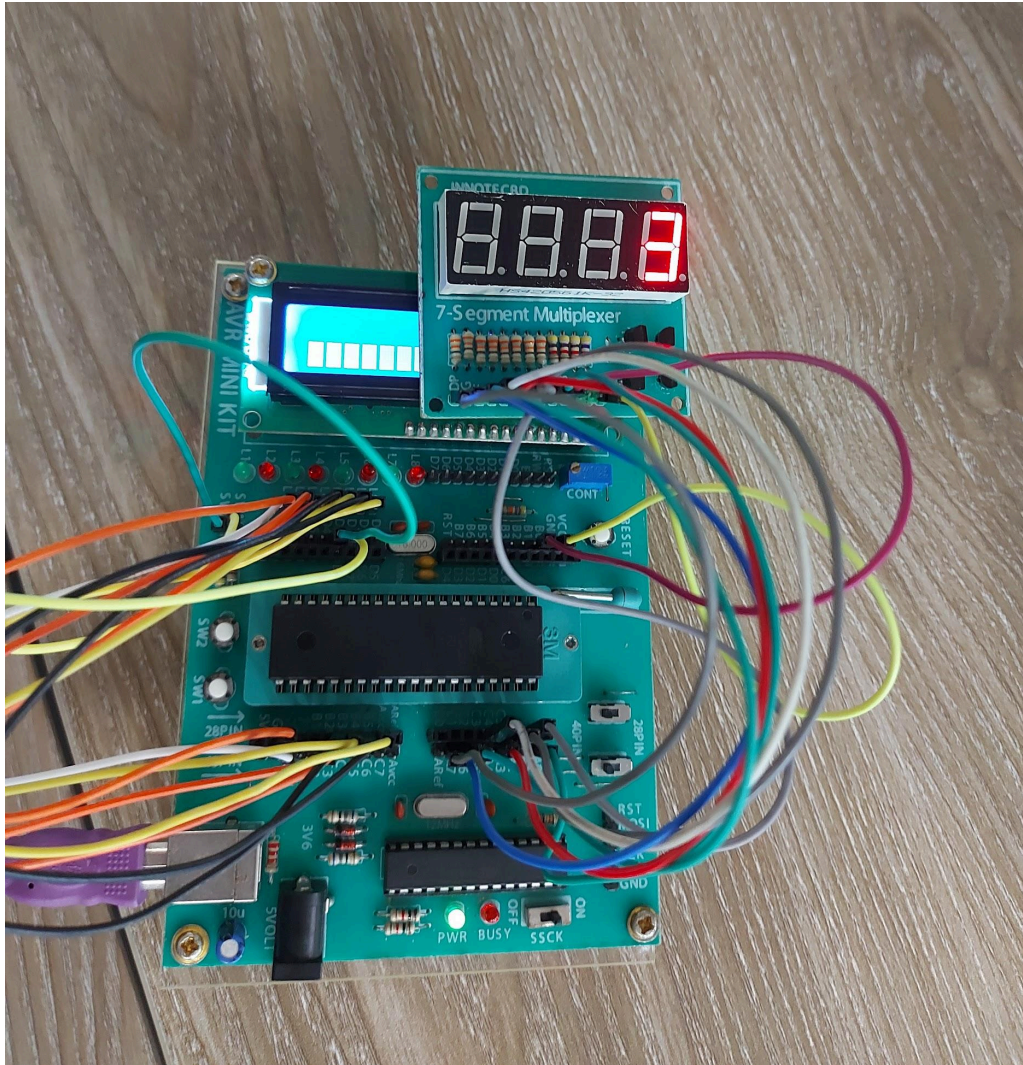
```

***Output***

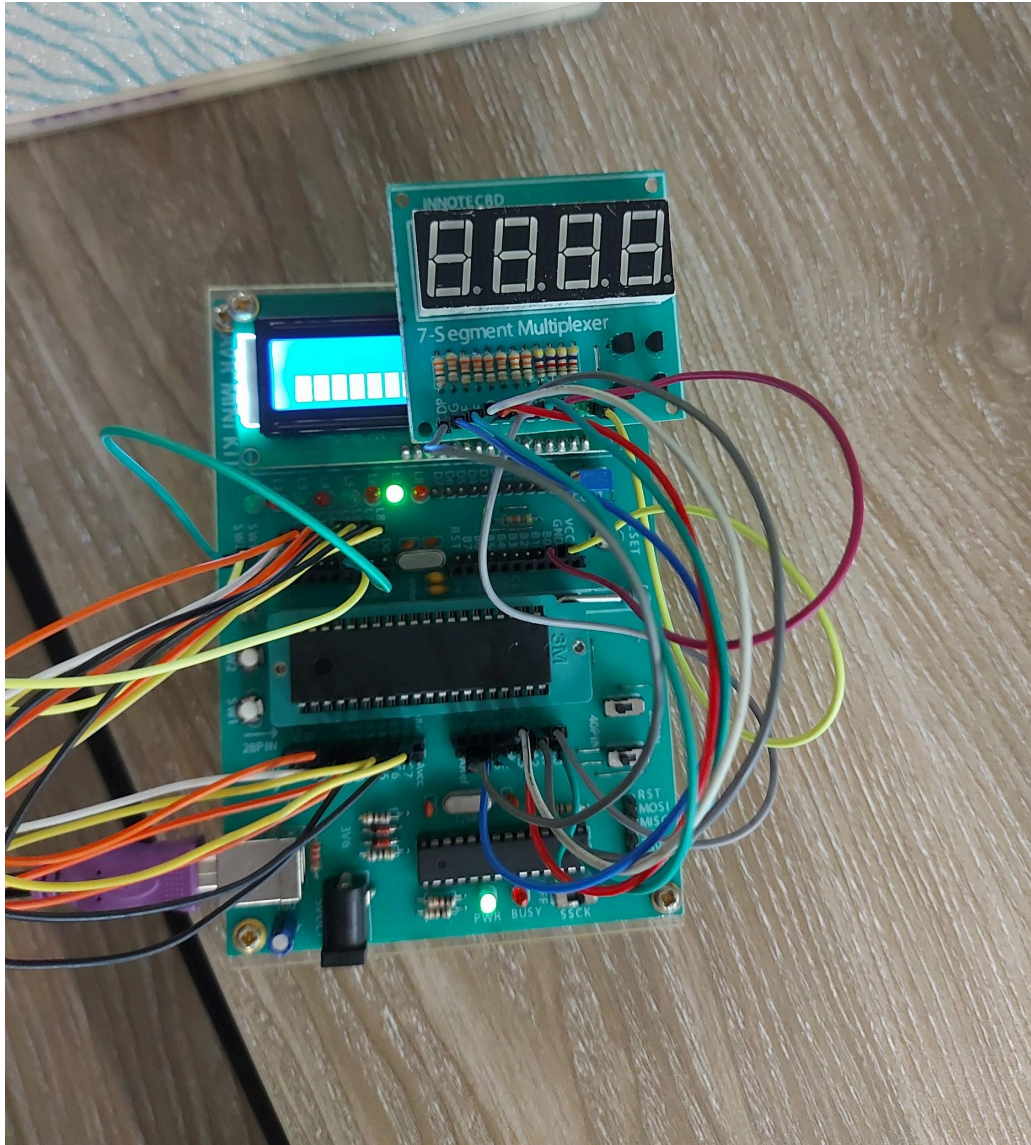


**Figure 05: When no Switch is pressed, nothing happens**



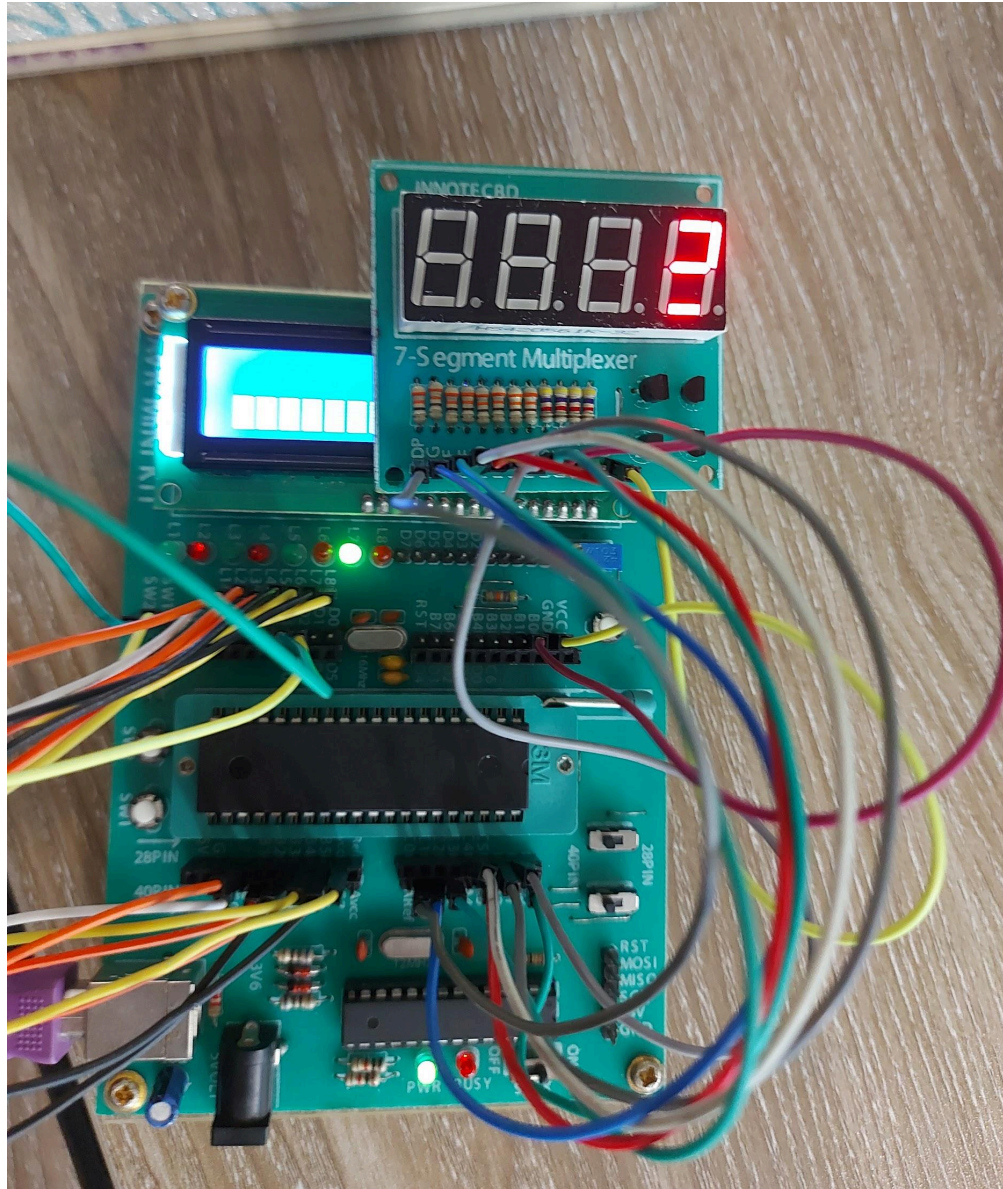


**Figure 06: When Switch 1 (Priority) is pressed, 7 Segment display shows countdown from 0-9**



**Figure 07: When Switch 2 is pressed, the led starts shifting**





**Figure 08:** When Switch 1 is pressed during the LED shifting, the LED shifting is interrupted and the 7-Segment display starts.

**Discussion:** This task covers multiple interrupts. For the interrupts, PORTD.2 (INT0) and PORTD.3(INT1) are used. Once the code is uploaded, nothing really happens because it's waiting for one of the switches to be pressed. When Switch 1 (INT0) is pressed, the 7-Segment display shows a countdown from 0-9 with a 1 second delay in between. During this process even if Switch 2 (INT1) is pressed, the task is not interrupted since INT0 is high priority. However when Switch 2 (INT1) is pressed, the LEDs start shifting and during this task if Switch 1(INT0 ) is pressed, the LED shifting is interrupted and paused and the 7-Segment display starts showing the count down and when that's over, the LED shifting task is resumed. Hence INT0 (7-Segment

task) has higher priority over INT1 (LED shifting) and to set this, the global interrupt, `#asm("sei")` is declared in the INT0 task to be the priority.

### *Lab Report Task*

Modify the code in such a way that will set INT1 higher than INT0

### *Code*

```
#include <mega32.h>
#include <delay.h>
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    unsigned int cathode[10]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
    unsigned int i=0,k=0;
    PORTB=0x01;
    #asm("sei")
    for (i=0;i<10;i++)
    {
        PORTA= cathode[i];
        delay_ms(1000);
    }
    PORTA=0;
    PORTB=0;
}
interrupt [EXT_INT1] void ext_int1_isr(void)
{
    char i;
    PORTC=0x80;
    for (i=0;i<8;i++)
    {
        delay_ms(1000);
        PORTC>>=1;
    }
}
void main(void)
{
    DDRA=0xFF;
    DDRC=0xFF;
    DDRB=0x01;
    PORTA=0;
    PORTC=0;
    PORTB=0;
    GICR|=(1<<INT1) | (1<<INT0) | (0<<INT2);
    MCUCR=(1<<ISC11) | (0<<ISC10) | (1<<ISC01) | (0<<ISC00);
```

```
MCUCSR=(0<<ISC2);
GIFR=(1<<INTF1) | (1<<INTF0) | (0<<INTF2);
asm("sei")
while (1)
{

}
}
```